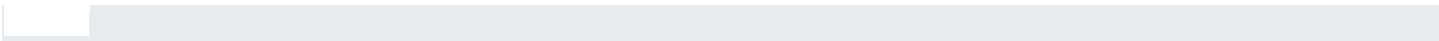


Table of Contents

- 1. Executive Summary
- 2. Vulnerability Overview
- 3. Detailed Findings
- 4. Remediation Guide
- 5. Methodology

1. Executive Summary

Overall Risk Score



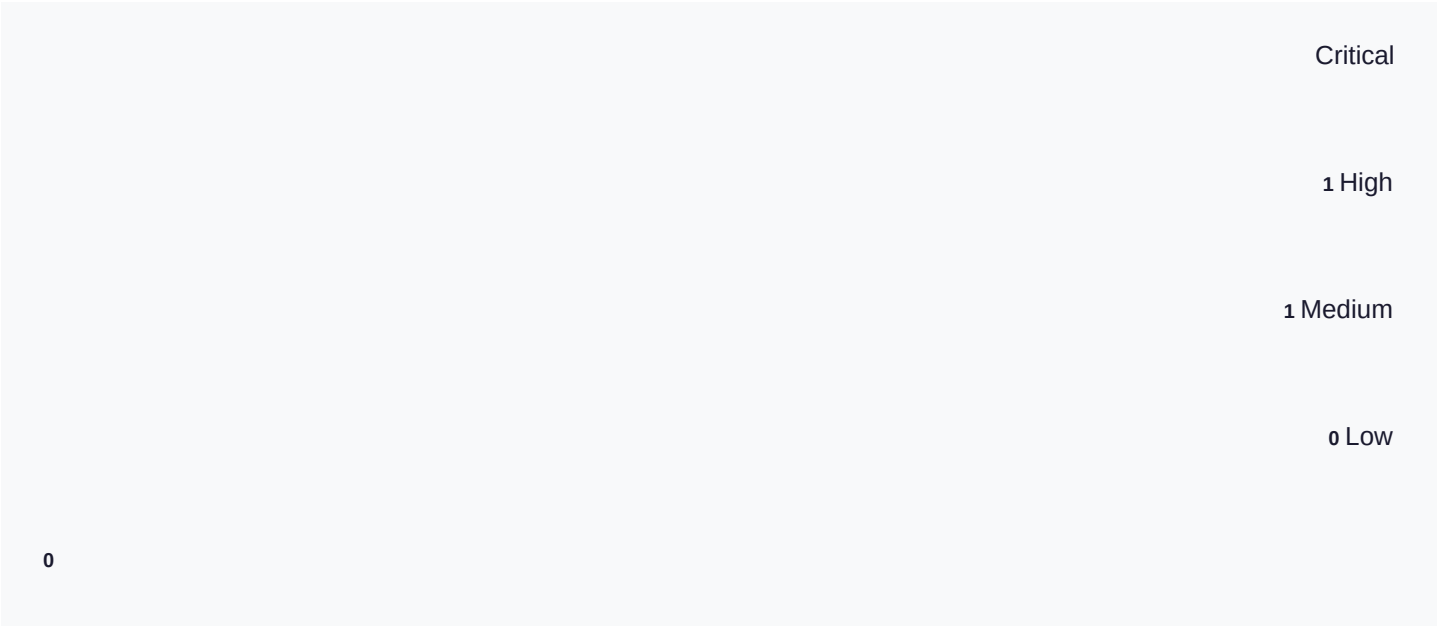
Medium risk - Some vulnerabilities require attention.

Property	Value
Target URL	https://example.com

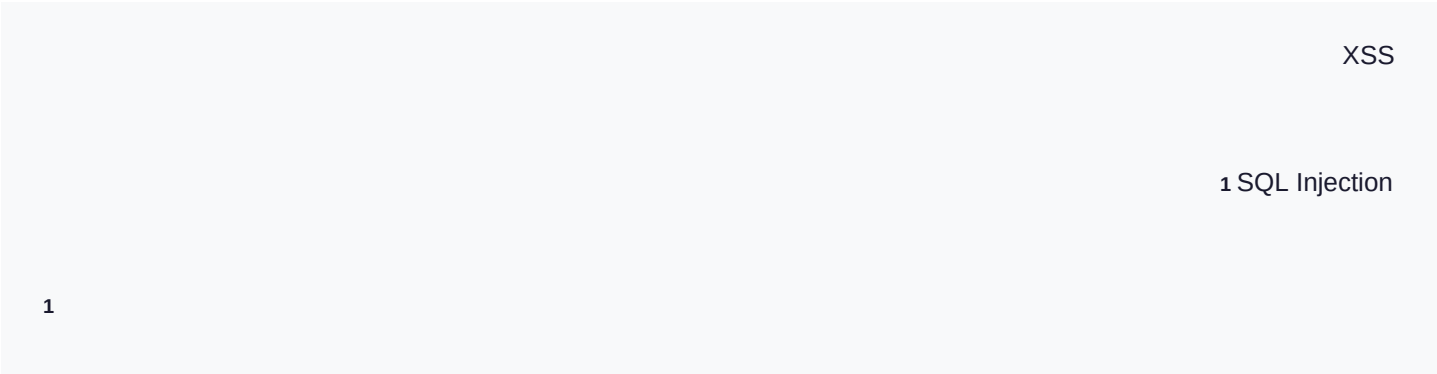
Scan Start	2024-01-28 10:00:00
Scan End	2024-01-28 10:30:00
Status	completed
Total Vulnerabilities	2

2. Vulnerability Overview

Severity Distribution



Vulnerability Types



Vulnerabilities Summary Table

#	Type	Severity	URL	Parameter	Confidence
1	XSS	HIGH	https://example.com/search		0%
2	SQL Injection	CRITICAL	https://example.com/users		0%

3. Detailed Findings

1. XSS

HIGH

Affected URL
https://example.com/search
Vulnerable Parameter
q
Payload Used
Evidence
Payload reflected in response

- How to Fix
- Encode all user input before displaying in HTML (use HTML entity encoding)
 - Use Content-Security-Policy (CSP) headers to restrict script sources
 - Implement HttpOnly and Secure flags on session cookies
 - Use modern frameworks that auto-escape output (React, Vue, Angular)
 - Validate and sanitize all input on the server side
 - UsetextContent instead ofinnerHTML when setting element content

2. SQL Injection

CRITICAL

Affected URL
https://example.com/users
Vulnerable Parameter
id
Payload Used

' OR 1=1 --

Evidence

SQL error in response

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

4. Remediation Guide

This section provides detailed guidance on how to fix the vulnerabilities found during the scan. Each vulnerability type includes an explanation of how the attack works and code examples for remediation.

XSS

1 Finding(s)

What is XSS?

Cross-Site Scripting (XSS) occurs when an application includes untrusted data in a web page without proper validation or escaping. This allows attackers to execute malicious scripts in victims' browsers.

How the Attack Works

1. Attacker identifies an input field that reflects user data in the page 2. Attacker crafts a malicious payload containing JavaScript code 3. When a victim views the page, the script executes in their browser 4. The script can steal cookies, session tokens, or perform actions as the user

Potential Impact

- Session hijacking and account takeover
- Theft of sensitive data (cookies, tokens)
- Defacement of web pages
- Phishing attacks using trusted domain
- Malware distribution

Remediation Steps

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use `textContent` instead of `innerHTML` when setting element content

Code Examples

VULNERABLE CODE:

SECURE CODE:

```
// Server-side (Python/Flask example): from markupsafe import escape @app.route('/display') def display(): user_input = request.args.get('input', '') safe_input = escape(user_input) return render_template('page.html', content=safe_input)
```

References

- <https://owasp.org/www-community/attacks/xss/>
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

CWE: CWE-79 OWASP: A03:2021 - Injection

SQL Injection

1 Finding(s)

What is SQL Injection?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact

- Complete database compromise
- Data theft (credentials, personal info, financial data)
- Data modification or deletion
- Authentication bypass
- Remote code execution (in some cases)
- Full server compromise

Remediation Steps

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query) # VULNERABLE CODE (PHP) $query = "SELECT * FROM users WHERE id = " .
$_GET['id']; $result = mysqli_query($conn, $query);
```

SECURE CODE:

```
# SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user =
session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) $stmt =
$pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute($_GET['id']);
```

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 OWASP: A03:2021 - Injection

5. Methodology

This automated scan was performed using the Web Vulnerability Scanner, which employs the following techniques to discover and test for vulnerabilities:

Scanning Phases

Phase	Description
1. Discovery	Crawls the target website to discover pages, forms, API endpoints, and parameters using browser automation (Playwright).
2. Analysis	Analyzes discovered elements to identify potential injection points and build a map of testable targets.
3. Testing	Tests each target with specialized payloads for various vulnerability types (XSS, SQLi, etc.).
4. Validation	Confirms findings using multi-stage validation to reduce false positives. Assigns confidence scores.

Vulnerability Types Tested

Cross-Site Scripting (XSS)	SQL Injection	NoSQL Injection	Path Traversal	Open Redirect	SSTI	Command Injection	SSRF	XXE	Header Injection
----------------------------	---------------	-----------------	----------------	---------------	------	-------------------	------	-----	------------------

Disclaimer: This automated scan may not detect all vulnerabilities. Manual penetration testing by qualified security professionals is recommended for comprehensive security assessment. Always obtain proper authorization before scanning.